

REACTJS INTERVIEW BLUEPRINT

200 Questions to Master
ReactJS

BY UPAMANYU DEKA

By Upamanyu Deka

ReactJS Interview Blueprint

About this Book

This book is your ****interview goldmine**** for mastering ReactJS — combining 150 conceptual questions and 50 hands-on machine coding challenges. Each question is crafted to reflect real interview scenarios at product-based companies.

It's designed to sharpen not just what you know, but ***how you explain it*** — the real skill that differentiates great candidates.

How to Use this Book – The 8-Week Blueprint

1■■ Week 1–2: Core React (JSX, Props, State, Hooks).

2■■ Week 3–4: Intermediate React (Context, Optimization, Lifecycle).

3■■ Week 5–6: Advanced React (Fiber, Concurrent Rendering, Suspense).

4■■ Week 7: Machine Coding (UI components, Hooks, Feature Builds).

5■■ Week 8: Mock and Revise — explain your answers out loud.

■ *Pro Tip:* Treat each question as a prompt for a real interview discussion. Focus on clarity, reasoning, and practical examples.

By Upamanyu Deka

ReactJS Interview Blueprint

200 Questions to Master ReactJS

Part A – ReactJS (150 Questions)

1. What is React and why is it popular?
2. What are components in React?
3. Difference between class and functional components?
4. What is JSX and how does it work?
5. What are props in React?
6. What is state and how is it used?
7. Difference between props and state?
8. What are React hooks?
9. Explain useState hook with example.
10. Explain useEffect hook and its dependencies.
11. What are controlled components?
12. What are uncontrolled components?
13. What is conditional rendering?
14. What are fragments in React?
15. What is the purpose of keys in React lists?
16. What is prop drilling?
17. What is React.memo and when to use it?
18. What is the difference between useMemo and useCallback?
19. What is context API and when should it be used?
20. What is lifting state up?
21. How do you handle forms in React?
22. Explain event handling in React.
23. What is a custom hook?
24. What is reconciliation in React?
25. What is Virtual DOM and how does it work?
26. Explain how React updates the real DOM efficiently.

27. What is `React.StrictMode`?
28. What are default props?
29. How to share data between sibling components?
30. What is the difference between mounting and updating phases?
31. What are pure components?
32. What is shallow comparison in React?
33. How do you optimize rendering performance in React?
34. What are higher-order components (HOCs)?
35. What are render props?
36. Explain `useRef` hook.
37. What is `useLayoutEffect`?
38. How does React handle events internally?
39. What are React portals?
40. How do you perform conditional rendering efficiently?
41. Explain React Fiber architecture.
42. How does React handle re-rendering efficiently?
43. What is batching in React?
44. How does React reconcile lists with keys?
45. What are uncontrolled vs controlled inputs?
46. What is lazy loading in React?
47. What is `React.Suspense` used for?
48. How do you handle side effects in React?
49. Explain error boundaries in React.
50. What is code splitting in React?
51. Explain React's diffing algorithm.
52. What are React fibers and lanes?
53. What is concurrent rendering?
54. What is hydration and how does it work?
55. SSR vs CSR vs ISR vs SSG — differences?
56. What is `useReducer` hook and when to use it?
57. What is state management in React?
58. How does Context API compare to Redux?

59. How do you optimize React app performance?
60. What is memoization in React?
61. How to prevent unnecessary re-renders?
62. What are Suspense boundaries?
63. What is server-side rendering (SSR)?
64. How does React handle asynchronous rendering?
65. What is event delegation in React?
66. How to handle long lists efficiently in React?
67. What is React Profiler?
68. How does useTransition improve performance?
69. What is startTransition and its benefits?
70. How to handle error states in React hooks?
71. What are the benefits of immutability in React?
72. How does React's synthetic event system work?
73. What are StrictMode double renders?
74. Explain the render and commit phases in React.
75. How does React manage memory during re-renders?
76. What is key collision in React lists?
77. What are some common React anti-patterns?
78. How does React handle forms with validation libraries?
79. What are the drawbacks of context API?
80. What is server component architecture?
81. How does Suspense integrate with data fetching libraries?
82. Explain the useSyncExternalStore hook.
83. What is React's concurrency model?
84. What is React hydration mismatch and how to fix it?
85. What is double buffering in React rendering?
86. How does React maintain UI responsiveness?
87. Explain the concept of time slicing.
88. What is cooperative scheduling?
89. How to measure React performance using Profiler API?
90. What is React Fiber tree reconciliation?

91. How does React coordinate updates between components?
92. What are transitions in React 18?
93. How to handle slow network rendering gracefully?
94. Explain React's commit priorities.
95. What is lazy initialization in useState?
96. How to manage derived state correctly?
97. How does React isolate side effects?
98. Explain batching strategy in concurrent mode.
99. How does React decide which components to re-render?
100. How does React handle layout thrashing?
101. What are React's internal linked lists (Fiber nodes)?
102. How does React implement cooperative multitasking?
103. Explain the architecture of React's scheduler.
104. What is React's diff optimization heuristic?
105. How do React hooks work internally?
106. How does React identify hook order consistency?
107. What are React lanes and priorities in concurrent mode?
108. How does React perform time slicing for rendering?
109. What is SuspenseList and how is it used?
110. How do Suspense boundaries affect data fetching?
111. Explain how React queues state updates internally.
112. What are the trade-offs of React's virtual DOM model?
113. How does React handle reconciliation at scale?
114. Explain how React batches updates in concurrent rendering.
115. How does React pause and resume rendering?
116. What are React's scheduler APIs (requestIdleCallback, postTask)?
117. How does React handle cross-component dependencies?
118. How does React ensure UI consistency across threads?
119. What happens in React's commit phase in detail?
120. Explain fiber reconciliation with examples.
121. How does React schedule priority-based updates?
122. How does React diff keyed lists efficiently?

123. What are microtask queues and how do they affect React updates?
 124. Explain the architecture of React Server Components.
 125. How does React isolate rendering work from user interaction?
 126. How does React optimize hydration performance?
 127. How does React handle memory leaks in async hooks?
 128. Explain Suspense's role in streaming SSR.
 129. How does React integrate with frameworks like Next.js?
 130. What is the difference between legacy and concurrent rendering?
 131. How does React manage deferred values?
 132. How does React handle component unmounts during rendering?
 133. How does React optimize repeated renders via memoization trees?
 134. What is React's bailout mechanism?
 135. How does React's Fiber tree store alternate versions?
 136. How does React integrate with non-React systems?
 137. How do Suspense and transitions interact?
 138. How to handle Suspense boundaries in nested trees?
 139. What are React's internal debugging tools?
 140. How does React track hook dependencies efficiently?
 141. Explain React's coordination between rendering and layout.
 142. How does React manage re-entrant rendering?
 143. What are React's internal object pools?
 144. How does React schedule IO and CPU-bound tasks?
 145. How does React detect infinite render loops?
 146. What are future improvements coming to React Fiber?
 147. How does React integrate with WebAssembly and Web Workers?
 148. How does React achieve atomic UI updates?
 149. What is a React WorkLoop and how does it operate?
 150. How does React manage partial hydration?
-

Part B – React Machine Coding (50 Challenges)

Easy

1. Build a Todo app with add/edit/delete and localStorage persistence.
2. Create an Accordion component (single/multiple open modes).
3. Implement a Modal with focus trap and ESC to close.
4. Create a Tabs component with keyboard navigation.
5. Build a Star Rating component (hover preview + controlled state).
6. Implement a Search box with debounced API calls.
7. Create a Pagination component (page numbers, next/prev, disabled states).
8. Build a Toggle (Switch) component with accessible labels.
9. Implement a Character Counter input with max length warning.
10. Create a Tooltip component with smart positioning.
11. Build an Image Carousel with autoplay and pause on hover.
12. Implement a Copy-to-Clipboard button with feedback toast.
13. Create a File Upload UI with drag-and-drop and preview.
14. Build a Stepper (multi-step form) with validation per step.
15. Implement a Countdown Timer with start/pause/reset.

Medium

16. Build a Typeahead (Autocomplete) using client-side search and keyboard nav.
17. Implement Infinite Scroll with IntersectionObserver and cancellation.
18. Create a Form Builder powered by a JSON schema (render inputs dynamically).
19. Build a Multi-Select Dropdown with tags and search inside the list.
20. Implement a Notification/Toast queue with auto-dismiss and manual close.
21. Create a Data Table with sorting, filtering, and pagination.
22. Build a Tree View (expand/collapse) with async child loading.
23. Implement a Kanban board with drag-and-drop and column limits.
24. Create a Calendar date picker with range selection and disabled dates.
25. Build a Chat UI: virtualized message list + input + typing indicator.
26. Implement an Image Gallery with lightbox and keyboard controls.
27. Build a Markdown Editor with live preview (split pane).
28. Implement Route Guards and protected routes (React Router).
29. Create a Global Search overlay (Cmd/Ctrl+K) with fuzzy search.
30. Build a Poll/Voting widget with optimistic updates.

31. Implement a Shopping Cart with computed totals and coupons.
32. Create a File Explorer (folders/files) with breadcrumb navigation.
33. Build a Custom Hook useDebounceValue and use it in a search UI.
34. Implement a Color Picker with HEX/RGB and copy support.
35. Create a Responsive Grid with draggable/resizable cards.

Hard

36. Implement a Virtualized List with dynamic row heights and windowing.
37. Build a Multi-level Sidebar Menu (Amazon-style) with hover intent.
38. Implement an Offline-first Notes app with sync and conflict resolution sketch.
39. Create a Form Engine with field-level validation, async rules, and dependencies.
40. Build a Collaborative Cursor demo using WebSockets (client-side logic).
41. Implement a Scheduler: limit parallel tasks, retry with backoff, and cancellation.
42. Create a Dashboard with widgets: lazy loading, suspense boundaries, and error boundaries.
43. Build a Drag-and-Drop file uploader with chunked uploads and retry logic (client only).
44. Implement a Diff Viewer for JSON objects highlighting adds/edits/deletes.
45. Create a Spreadsheet-like grid with formula cells and dependency recomputation.
46. Implement a Real-time Search with abortable fetch and stale-while-revalidate cache.
47. Build a Route-level Code Splitting demo with prefetching on hover/intent.
48. Implement a Performance Monitor overlay (FPS, long tasks, memory snapshot).
49. Create an Image Editor (crop/rotate/scale) with Web Workers for heavy ops.
50. Build a Feature Flag demo with remote config and rollout percentages (client logic).